

CASE-004

Carga Inicial de Clientes e Contas

Diagnostico, comparativo e licoes aprendidas sobre a correcao do pipeline de carga inicial do laboratorio bancario mainframe.

Artefatos envolvidos:

- EBCOMP.jcl - Compilacao COBOL
- EBJCLLD.jcl - Execucao da carga
- EBCLOAD.cbl - Programa de carga
- clientes.txt - Dados seed de clientes
- contas.txt - Dados seed de contas

Projeto	Emunah Bank Lab
Ambiente	IBM zXplore (z/OS)
HLQ	<HLQ>.EMUNAH.*
Ferramentas	Zowe CLI, COBOL, JCL, VSAM, IDCAMS
Case	004 - Correcao do Pipeline de Carga Inicial

INDICE

1. Resumo Executivo
2. Visao Bancaria
3. Visao Mainframe
 - 3.1 EBCOMP.jcl – Compilacao
 - 3.2 EBJCLLD.jcl – JCL de Carga
 - 3.3 EBCLLOAD.cbl – Programa COBOL
 - 3.4 Dados Seed – clientes.txt e contas.txt
4. Comparativo Antes x Depois (Tabelas)
5. Riscos Operacionais Resolvidos
6. Licoes Aprendidas
7. Aplicacao no Emunah Lab
8. Proximos Passos
9. Resumo em 1 Paragrafo

ANOTAÇÕES – OBJETIVOS DE ESTUDO

1. RESUMO EXECUTIVO

Foram implementadas tres mudancas coordenadas que corrigiram o pipeline de carga inicial do Emunah Bank Lab, elevando o realismo operacional e a seguranca do processamento batch:

Artefato	Mudanca Principal	Objetivo
EBCOMP.jcl	Migrar para procedure IGYWCL	Eliminar S806, automatizar link-edicao
EBJCLLD.jcl	Adicionar STEP0 IDCAMS + mudar DISP=SHR para OLD	Garantir KSDS vazio e acesso exclusivo
EBCLLOAD.cbl	Nomes proprios nos FDs + file status apos I/O	Evitar ambiguidade, diagnosticar erros reais

Cada mudanca resolve um problema silencioso que bloqueava a execucao realista. Juntas, transformam o pipeline de fragil e silencioso para robusto e diagnostico.

ANOTACOES

2. VISAO BANCARIA

Problema bancario resolvido

Em um banco real, a carga inicial de dados e operacao critica que deve:

- Comecar de estado conhecido – KSDS vazio, sem lixo de execucoes anteriores
- Garantir integridade referencial – nenhum cliente orfao, nenhuma conta sem cliente
- Deixar auditoria clara – por que cada rejeicao aconteceu (status invalido, duplicado, overflow)
- Nao perder registros silenciosamente – se gravou zero registros, o operador deve saber

O risco que foi mitigado

■ **ATENCAO:** Antes: Se DISP=SHR fosse usado em WRITE num KSDS com OPEN I-O, o sistema deixaria a operacao "bem-sucedida" (RC=0) mas nao gravava nada. O operador nao via aviso algum; veria um resumo dizendo "1000 gravados" quando na verdade 0 foram escritos no arquivo.

Depois, a combinacao de:

- STEP0 DELETE/DEFINE garante arquivo vazio e conhecido
- DISP=OLD forca acesso exclusivo (bloqueia concorrencia)
- OPEN OUTPUT e inequivoco (carga, nao atualizacao)
- File status apos WRITE diz exatamente o que deu errado

ANOTACOES

3. VISAO MAINFRAME

3.1 EBCOMP.jcl – Migracao para IGYWCL

■ ANTES

```
//STEP1 EXEC PGM=IGYCRCTL  
//SYSIN DD DSN=...EBCLLOAD,DISP=SHR  
//SYSLIB DD DSN=...COPY,DISP=SHR  
//SYSLIN DD DSN=&&OBJSET,DISP=(,PASS),...  
//SYSPRINT DD SYSOUT=*
```

■ DEPOIS

```
//STEP1 EXEC IGYWCL  
//COBOL.SYSIN DD DSN=...EBCLLOAD,DISP=SHR  
//COBOL.SYSLIB DD DSN=...COPY,DISP=SHR  
//LKED.SYSLMOD DD DSN=...LOADLIB(EBCLLOAD),DISP=SHR  
//LKED.SYSPRINT DD SYSOUT=*
```

Aspecto	ANTES	DEPOIS	Por que
Compiler	Chamado direto (PGM=IGYCRCTL)	Via procedure catalogada (IGYWCL)	STEPLIB pre-configurado pela IBM
Link-edit	Manual, num job separado (EBLINK)	Automatico dentro de IGYWCL	Reduz steps, menos chance de erro
DDNAMEs	Generico (SYSIN, SYSLIN)	Qualificado (COBOL.SYSIN, LKED.*)	Clareza, namespace isolado
Manutencao	Trocas de STEPLIB por versao	Procedure ja mantida pela IBM	Menor impacto de atualizacoes

Erro resolvido: S806 (modulo nao encontrado) – porque IGYCRCTL precisa estar no LNKLIST ou ter STEPLIB explicito. A procedure IGYWCL ja traz isso.

ANOTACOES

3.2 EBJCLLD.jcl – IDCAMS + DISP=OLD

■ ANTES

```
//STEP1 EXEC PGM=EBCLLOAD
//CLIENTE DD DSN=...CLIENTE.KSDS,DISP=SHR
//CONTA DD DSN=...CONTA.KSDS,DISP=SHR
```

■ DEPOIS

```
//STEP0 EXEC PGM=IDCAMS
//SYSIN DD *
DELETE ...CLIENTE.KSDS CLUSTER PURGE
SET MAXCC=0
DEFINE CLUSTER (NAME(...) KEYS(5 0) RECORDSIZE(80 80) ...)
DELETE ...CONTA.KSDS CLUSTER PURGE
SET MAXCC=0
DEFINE CLUSTER (NAME(...) KEYS(12 0) RECORDSIZE(100 100) ...)
/*
//STEP1 EXEC PGM=EBCLLOAD
//CLIENTE DD DSN=...CLIENTE.KSDS,DISP=OLD
//CONTA DD DSN=...CONTA.KSDS,DISP=OLD
```

Aspecto	Impacto
DELETE + DEFINE	Reseta KSDS para estado vazio conhecido; evita dados "fantasma" de execucao anterior
SET MAXCC=0	Permite prosseguir mesmo se DELETE falhar (primeira execucao, arquivo ainda nao existe)
KEYS e RECORDSIZE	Devem casar com o copybook: KEYS(5 0) = CLI-ID 5 digitos KEYS(12 0) = AG(4) + CTA(8) RECORDSIZE(80 80) e (100 100)
DISP=SHR -> DISP=OLD	CRITICO: KSDS com OPEN I-O e DISP=SHR permite OPEN bem-sucedido mas bloqueia WRITES silenciosamente

Conceito mainframe: VSAM e dataset gerenciado. Precisa de IDCAMS para ciclo de vida (definicao, alocao, exclusao). Nao e como arquivo sequencial que voce cria com DD generico.

ANOTACOES

3.3 EBCLLOAD.cbl – FDs Proprios + File Status

Mudanca 1: FDs com nomes propios (nao COPY)

■ ANTES

```
FD CLIENTE-KSDS.  
01 CLIENTE-KSDS-REG.  
COPY CPCLI001. *> mesmo copybook da entrada
```

■ DEPOIS

```
FD CLIENTE-KSDS.  
01 CLIENTE-KSDS-REG.  
05 KCLI-CHAVE PIC 9(05).  
05 KCLI-RESTO PIC X(75).
```

Quando voce usa COPY do mesmo copybook em dois FDs (entrada e KSDS), ambos tem CLI-ID-CLIENTE como campo. Isso cria ambiguidade – o compilador fica em duvida sobre qual CLI-ID-CLIENTE voce quer quando escreve uma condicao. Nomes propios (KCLI-*, KCNT-*) eliminam a ambiguidade e deixam claro: estou manipulando o KSDS, nao a entrada.

Mudanca 2: File Status em vez de INVALID KEY

■ ANTES

```
WRITE CLIENTE-KSDS-REG  
INVALID KEY  
ADD 1 TO WS-CLI-REJEITADOS  
... auditoria ...  
NOT INVALID KEY  
ADD 1 TO WS-CLI-GRAVADOS  
END-WRITE.
```

■ DEPOIS

```
WRITE CLIENTE-KSDS-REG  
IF WS-FS-CLIENTE NOT = '00'  
ADD 1 TO WS-CLI-REJEITADOS  
STRING 'CLIENTE REJEITADO - FS='  
WS-FS-CLIENTE ...  
WRITE AUDIT-REG  
ELSE  
ADD 1 TO WS-CLI-GRAVADOS  
END-IF.
```

Detalhe	ANTES	DEPOIS	Motivo
Captura	Apenas duplicado (INVALID KEY)	File status diz tipo (21=dup, 24=overflow, 22=formato)	Diagnostico preciso
Output	Generico "duplicado"	"FS=21" ou "FS=24" na auditoria	Auditoria investigavel
Robustez	Se houver ambiguidade, INVALID KEY pode nao funcionar	File status sempre funciona	Independente de nomes

Mudanca 3: OPEN OUTPUT + ACCESS SEQUENTIAL + validacao de OPEN

Aspecto	ANTES	DEPOIS
OPEN	OPEN I-O	OPEN OUTPUT
ACCESS MODE	DYNAMIC	SEQUENTIAL

Aspecto	ANTES	DEPOIS
Validacao OPEN	Nao havia	IF WS-FS-CLIENTE NOT = '00' DISPLAY + MOVE 8 + GOBACK
Semantica	Manutencao/update	Carga inicial em vazio

ANOTACOES

3.4 Dados Seed – clientes.txt e contas.txt

clientes.txt (20 registros, LRECL=80)

Layout: CLI-ID(5) + CLI-NOME(30) + CLI-CPF(11) + CLI-DT-NASC(8) + CLI-STATUS(1) + CLI-DT-CAD(8) + FILLER. Registros vão de 00001 a 00020. Não houve mudança estrutural significativa – o DEPOIS apenas padronizou o LRECL com espaços no final para garantir exatamente 80 bytes por registro.

Exemplo – clientes.txt

```
00001JOAO SILVA 1234567890119900101A20260312
00002MARIA SOUZA 1234567890219850215A20260312
... (20 registros, todos com STATUS=A)
```

contas.txt – Mudança significativa

■ ANTES

```
Layout ANTES: 42 bytes por registro (não batia com LRECL=100)
000100000000100001CCA000000010050020260312
Campos: AG(4)+CTA(8)+IDCLI(5)+TIPO(2)+STATUS(1)+SALDO(13)+LIMITE(?)+DT
Problema: 20 registros, 1 conta por cliente, STATUS='A'
mas campo CNT-STATUS caindo na posição errada
```

■ DEPOIS

```
Layout DEPOIS: 100 bytes por registro (casa com RECORDSIZE(100 100))
000100000000100001CA20260312 0000000110000 00005000000
Campos: AG(4)+CTA(8)+IDCLI(5)+TIPO(1)+STATUS(1)+DT(8)+SALDO(13)+LIMITE(10)+FILLER
Agora: 40 registros, 2 contas por cliente (CC e PP), todos STATUS='A'
Layout coerente com copybook CPCNT001
```

O ANTES tinha um desalinhamento crítico: o registro de 42 bytes não preenchia o LRECL de 100 bytes do KSDS, e os campos de status ficavam em posições erradas em relação ao copybook CPCNT001. O DEPOIS corrigiu o layout para casar exatamente com a DEFINE do IDCAMS (RECORDSIZE(100 100)) e com o copybook, além de aumentar a massa de 20 para 40 contas (corrente + poupança por cliente).

ANOTAÇÕES

4. COMPARATIVO CONSOLIDADO

Cenario	Antes	Depois
Aluno executa EBCOMP primeira vez	Pode falhar com S806	Sempre funciona (IGYWCL cuida disso)
Aluno executa EBJCLLD duas vezes seguidas	Dados da 1a tentativa contaminam 2a	STEP0 limpa tudo, recomeca do zero
Carga de 20 clientes retorna RC=0	Aluno nao sabe se gravou ou nao	SYSOUT mostra "GRAVADOS: 20" com precisao
Um cliente e rejeitado	Auditoria diz apenas "REJEITADO"	Auditoria diz "FS=21" (duplicado) ou "FS=24" (overflow)
Registros de contas com layout errado	Rejeicao em massa sem diagnostico claro	Layout casa com copybook, carga limpa

ANOTACOES

5. RISCOS OPERACIONAIS RESOLVIDOS

Risk-1: Erro silencioso em gravacao VSAM

ANTES	DISP=SHR + OPEN I-O em KSDS permitia ler mas bloqueava gravacao sem erro visivel. WRITE aparenta sucesso, RC=0, COBOL acha que gravou, arquivo fica com 0 registros, operador e enganado.
DEPOIS	DISP=OLD forca I/O exclusivo; STEP0 DELETE/DEFINE garante estado limpo. WRITE tem acesso garantido, FS=00 confirma, arquivo contem registros reais.

Risk-2: Contaminacao de dados entre execucoes

ANTES	Rodar EBJCLLD duas vezes criava mistura de dados novos com antigos.
DEPOIS	STEP0 DELETE purga tudo antes de cada execucao.

Risk-3: Erro de compilacao nao diagnosticado

ANTES	Se IGYCRCTL nao estivesse no LNKLIST, S806 aparecia sem explicacao clara.
DEPOIS	IGYWCL (procedure catalogada IBM) encapsula tudo – STEPLIB, compiler, linker.

Risk-4: Ambiguidade em referencia de campos

ANTES	COPY CPCLI001 em dois FDs diferentes criava nomes duplicados. O compilador nao conseguia desambiguar CLI-ID-CLIENTE.
DEPOIS	Nomes proprios (KCLI-*, KCNT-*) eliminam ambiguidade completamente.

ANOTACOES

6. LICOES APRENDIDAS

Licao 1: Procedures Catalogadas sao Investimento

Use procedures IBM (IGYWCL, IGYSQL, etc) em vez de recriar passos. IBM mantém STEPLIB, environment setup, defaults. Voce ganha com updates sem mexer no JCL.

Aplicacao no lab: Sempre que disponivel, use procedure catalogada antes de chamar PGM direto.

Licao 2: VSAM Exige Gestao Explicita com IDCAMS

VSAM nao e dataset sequencial. Precisa de ciclo de vida (define, delete, alter). Cada job que usa KSDS deve ter STEP0 IDCAMS que DELETE (com SET MAXCC=0) e DEFINE com KEYS e RECORDSIZE que casem com o programa.

Aplicacao no lab: STEP0 IDCAMS e padrao agora em todos os jobs de carga/reload.

Licao 3: DISP=SHR em VSAM com I-O e Armadilha

DISP=SHR permite abrir mas nao escrever em KSDS com OPEN I-O. Sintoma: RC=0, "GRAVADOS: N" no SYSOUT, mas arquivo fica vazio. Solucao: Para leitura: DISP=SHR + OPEN INPUT. Para escrita: DISP=OLD + OPEN OUTPUT ou I-O.

Aplicacao no lab: Todos os DDNames de KSDS em jobs de gravacao usam DISP=OLD.

Licao 4: File Status e Diagnostico Preciso

Nunca dependa so de INVALID KEY. Use file status apos I/O. FS=00 sucesso | FS=21 duplicado | FS=22 formato | FS=24 overflow | FS=92 arquivo em uso

Aplicacao no lab: EBCLLOAD agora loga file status de cada WRITE na auditoria.

Licao 5: Ambiguidade de Nomes Causa Bugs Sutis

Em COBOL, se dois FDs usam COPY do mesmo copybook, o compilador nao consegue desambiguar referencias. Use estruturas proprias no FD do arquivo gerenciado (KSDS).

Aplicacao no lab: Todos os FDs de KSDS agora tem estrutura propria (K* para keys, R* para resto).

Licao 6: Auditoria Precisa > Contadores Genericos

Antes: "CLIENTE REJEITADO - DUPLICADO". Depois: "CLIENTE REJEITADO - FS=21 - ID=00123". Impacto: Auditoria investigavel, trace de erro, reprocessamento com confianca.

Aplicacao no lab: Toda rejeicao registra file status + identificador do registro.

Licao 7: Carga inicial nao e update

Se a intencao e popular arquivos do zero, o desenho deve refletir isso: KSDS vazio, DELETE/DEFINE, OPEN OUTPUT, acesso compativel com escrita sequencial.

Aplicacao no lab: Pipeline de carga sempre comeca com IDCAMS + OPEN OUTPUT.

Licao 8: Coerencia entre camadas e tudo

O job, o programa, o dataset e o tipo de processamento precisam contar a mesma historia. Quando cada camada "acha" que o cenario e outro, aparecem os bugs mais frustrantes.

Aplicacao no lab: Revisao cruzada JCL x COBOL x DEFINE x seed antes de submeter.

ANOTACOES

7. APLICACAO NO EMUNAH LAB

Estado ANTES (fragil)

- ✗ EBCOMP.jcl chamava IGYCRCTL diretamente → S806 em alguns ambientes
- ✗ EBJCLLD.jcl nao limpava KSDS → dados antigos misturados com novos
- ✗ DISP=SHR nos KSDS → ilusao de sucesso (RC=0 mas 0 registros gravados)
- ✗ EBCLOAD.cbl usava COPY ambiguo → risco de branch errado em validacao
- ✗ contas.txt com layout de 42 bytes → nao casava com RECORDSIZE(100 100)

Estado DEPOIS (robusto)

- ✓ EBCOMP usa IGYWCL → compile + link automatico, STEPLIB garantido
- ✓ EBJCLLD executa STEP0 IDCAMS → KSDS limpo antes de cada carga
- ✓ DISP=OLD garante acesso exclusivo → nenhuma escrita silenciosa
- ✓ EBCLOAD tem nomes proprios → zero ambiguidade, file status diagnostico
- ✓ contas.txt com 100 bytes → layout casa com copybook e DEFINE

Causa-raiz (em uma frase)

Voce tinha um fluxo de **carga inicial** sendo executado com artefatos ainda desenhados como se fossem de **manutencao/update generico**. Esse desalinhamento gerava o sintoma: "le, parece processar, mas os dados nao ficam gravados corretamente."

ANOTACOES

8. PROXIMOS PASSOS

Acoes imediatas

Documentacao

Atualizar 06-runbooks.md com:

- Sempre use IGYWCL para compilacoes COBOL
- Todo job de KSDS comeca com STEP0 IDCAMS DELETE/DEFINE
- DISP=OLD obrigatorio para escrita em VSAM

Template

Criar jcl/templates/TEMPLATE-LOAD-KSDS.jcl:

- STEP0 IDCAMS generico reutilizavel
- Comentarios sobre KEYS, RECORDSIZE, SHAREOPTIONS

Testes

Validar que EBJCLLD agora:

- Limpa KSDS antes de carregar
- Carrega 100% dos registros validos
- Rejeita com diagnostico preciso

Proximos programas

EBVALI01, EBPOST01, EBSALD01 – aplicar mesmos padroes:

- FDs com nomes proprios
- File status apos I/O critico
- IDCAMS STEP0 em jobs de reload

ANOTACOES – MEUS PROXIMOS PASSOS

9. RESUMO EM 1 PARAGRAFO

VSAM nao e arquivo – e dataset gerenciado que exige IDCAMS para ciclo de vida e acesso explicito (DISP=OLD para escrita). Procedures catalogadas (IGYWCL) eliminam erro de compilacao. File status diagnostica erro real, nao apenas "invalido". Estruturas de FD proprias (nao COPY) evitam ambiguidade. Auditoria com FS preciso (nao generica) torna incidentes investigaveis. Layout dos dados seed deve casar exatamente com RECORDSIZE e copybook. Juntas, essas mudancas transformam o pipeline de fragil e silencioso para robusto e diagnostico.

Referencia Rapida – File Status Codes VSAM

FS	Significado	Acao
00	Sucesso	Prosseguir normalmente
02	Duplicado em alternate key	Verificar se aceitavel
10	Fim de arquivo (AT END)	Encerrar leitura
21	Chave fora de sequencia ou duplicada	Rejeitar, logar na auditoria
22	Chave duplicada em escrita primaria	Rejeitar, logar na auditoria
24	Overflow – dataset cheio ou boundary error	Expandir DEFINE, alertar operacao
35	Arquivo nao encontrado	Verificar DEFINE e DD
39	Conflito de atributos entre JCL e programa	Verificar RECORDSIZE, KEYS, LRECL
92	Violacao de acesso (arquivo em uso)	Verificar DISP, outro job usando